# Chapter Two
# Problem Analysis

## 2.1 Input Analysis

The system reads input only from the console keyboard. There is no file input, no network input and no stored configuration. Nine distinct inputs exist across the six menu options.

1. **Menu choice**
    - **Where entered:** main menu
    - **Data type and size:** unsigned byte
    - **Valid range:** 1 to 6
    - **Validation rule:** must be an integer from 1 to 6
    - **Error response:** show "Invalid choice" and redisplay the menu
2. **Customer name**
    - **Where entered:** Book New Ticket
    - **Data type and size:** ASCII string, 1 byte per character, stored in a 21-byte field (20 characters plus a null terminator)
    - **Valid range:** 1–20 characters
    - **Validation rule:** must not be empty and must not exceed 20 characters
    - **Error response:** show a name error and re-prompt
3. **Booking ID (new)**
    - **Where entered:** Book New Ticket
    - **Data type and size:** unsigned doubleword
    - **Valid range:** 100000–999999
    - **Validation rule:** must be exactly six digits in range, and must not match the ID of any **active** record
    - **Error response:** out of range, re-prompt; duplicate, "ERROR: Booking ID already exists!"
4. **Movie code**
    - **Where entered:** Book New Ticket
    - **Data type and size:** unsigned byte
    - **Valid range:** 1 to 3
    - **Validation rule:** 1 = Movie A, 2 = Movie B, 3 = Movie C
    - **Error response:** show a movie code error and re-prompt
5. **Seat number**
    - **Where entered:** Book New Ticket
    - **Data type and size:** unsigned byte
    - **Valid range:** 1 to 20

- **Validation rule:** must be in range **and** currently free in the seat map of the chosen movie
- **Error response:** out of range, re-prompt; taken, "ERROR: Seat already reserved!"

6. **Ticket category**
   - **Where entered:** Book New Ticket
   - **Data type and size:** unsigned byte
   - **Valid range:** 1 to 3
   - **Validation rule:** 1 = Adult, 2 = Student, 3 = Child
   - **Error response:** show a category error and re-prompt
7. **Booking ID (cancel)**
   - **Where entered:** Cancel Booking
   - **Data type and size:** unsigned doubleword
   - **Valid range:** 100000–999999
   - **Validation rule:** must match an **active** record
   - **Error response:** "ERROR: Booking ID not found!"
8. **Booking ID (display)**
   - **Where entered:** Display Ticket
   - **Data type and size:** unsigned doubleword
   - **Valid range:** 100000–999999
   - **Validation rule:** must match an **active** record
   - **Error response:** "ERROR: Booking ID not found!"
9. **Movie code (seat map)**
   - **Where entered:** Display Seating Map
   - **Data type and size:** unsigned byte
   - **Valid range:** 1 to 3
   - **Validation rule:** 1 to 3 only
   - **Error response:** show a movie code error and re-prompt

## Why these data sizes

The sizes above follow directly from the integer storage sizes studied in Chapter One:

- A **byte** stores an unsigned value from **0 to 255**. This covers the menu choice (1 to 6), the movie code (1 to 3), the seat number (1 to 20), the ticket category (1 to 3) and the active flag (0 or 1) with a large margin, so a byte is the correct and cheapest choice for all five.

- A **word** stores an unsigned value from **0 to 65,535**. This is **too small for a booking ID**, because the largest legal ID, 999,999, is far beyond 65,535 and would overflow the field.
- A **doubleword** stores an unsigned value from **0 to 4,294,967,295**, which holds any six-digit booking ID comfortably. Prices and revenue totals are also kept in doublewords so that all arithmetic operands share one width.
- The customer name is stored as **ASCII characters, one byte per character**, as covered in the character-storage topic of Chapter One. The name field is 21 bytes: 20 character codes plus one byte for the terminating null that marks the end of the string.

## Stored inputs and transient inputs

The width of an input is decided by the range of values it must hold, and nothing else. A value is never given a wider field just because it is typed at the keyboard.

Four of the nine inputs are **stored**: the customer name, the booking ID, the movie code, the seat number and the ticket category are written into a booking record and must stay there for the life of the booking. Their widths add up to the record size, so every byte counts.

The remaining inputs are **transient**. The menu choice, the booking ID used to cancel or to display a ticket, and the movie code used to pick a seating map are read, checked, acted upon and then discarded. None of them occupies space in a record. Even so, each is sized by its own range: the menu choice and the seat-map movie code fit in a byte, while a transient booking ID still needs a doubleword, because 999,999 overflows a word no matter how briefly the value is held.

## Capacity checks before a booking is accepted

Two further conditions are tested during booking. They are not typed by the user, but a booking is rejected if either fails:

1. **System capacity:** the system holds at most **60 active bookings**. An attempt to create the 61st produces `"ERROR: Cinema capacity reached!"`.
2. **Hall capacity:** each movie has **20 seats**. When all 20 seats of a movie are reserved, that movie is full and no further seat in it can be chosen.

## Non-numeric input

Every numeric prompt (menu choice, booking ID, seat number, movie code, ticket category) checks that the entered characters are digits before any range check runs. A prompt that receives a non-digit character shows the same error as an out-of-range value and re-reads the field, so a stray letter can never reach the range or lookup logic below it.

## 2.2 Output Analysis

All output is formatted console text. Each output is produced on demand from the data currently held in memory; nothing is written to a file.

1. **Main menu**
   - **Trigger:** program start and after every completed operation
   - **Format:** banner line, six numbered options, input prompt
   - **Data source:** fixed text
2. **Input prompts**
   - **Trigger:** during every operation
   - **Format:** one short line per field, stating the legal range
   - **Data source:** fixed text
3. **Booking confirmation**
   - **Trigger:** a booking is accepted
   - **Format:** `Booking added successfully!` followed by `Final Ticket Price: n JOD`
   - **Data source:** final price just calculated
4. **Formatted ticket**
   - **Trigger:** menu option 3 with a valid active ID
   - **Format:** banner, customer name, booking ID, movie, seat, category, base price, discount, separator line, final price
   - **Data source:** one active booking record
5. **Seating map**
   - **Trigger:** menu option 4 with a valid movie code
   - **Format:** title, legend `[O] Available [X] Reserved`, 4 rows of 5 seats, reserved count, occupancy percentage
   - **Data source:** seat map of the selected movie
6. **Per-movie statistics**
   - **Trigger:** menu option 5
   - **Format:** one line per movie: bookings, revenue in JOD, occupancy percentage
   - **Data source:** per-movie counters and revenue totals
7. **Overall totals**
   - **Trigger:** menu option 5
   - **Format:** total active bookings, overall revenue, average ticket price

- **Data source:** totals accumulated over the active records

8. **Most and least booked movie**
   - **Trigger:** menu option 5
   - **Format:** two labelled lines naming one movie each
   - **Data source:** the three per-movie booking counts
9. **Category histogram**
   - **Trigger:** menu option 5
   - **Format:** one row per category, a run of asterisks, and the count in brackets
   - **Data source:** the three category counters
10. **Error messages**
    - **Trigger:** any failed validation
    - **Format:** a single line beginning `ERROR:` where specified, otherwise a short explanatory line, followed by a re-prompt
    - **Data source:** fixed text

All ten outputs are plain console text produced through **library calls**, which is the highest of the four input-output levels studied in Chapter Two. The program does not call BIOS routines and does not address hardware ports directly. This is what keeps the program portable across lab machines, and it is the reason graphics, colour control and printer output are excluded from the project scope.

## 2.3 Process Flow

### 2.3.1 Main program flow

The program initialises its data once, then repeats a menu cycle until the user chooses to exit.

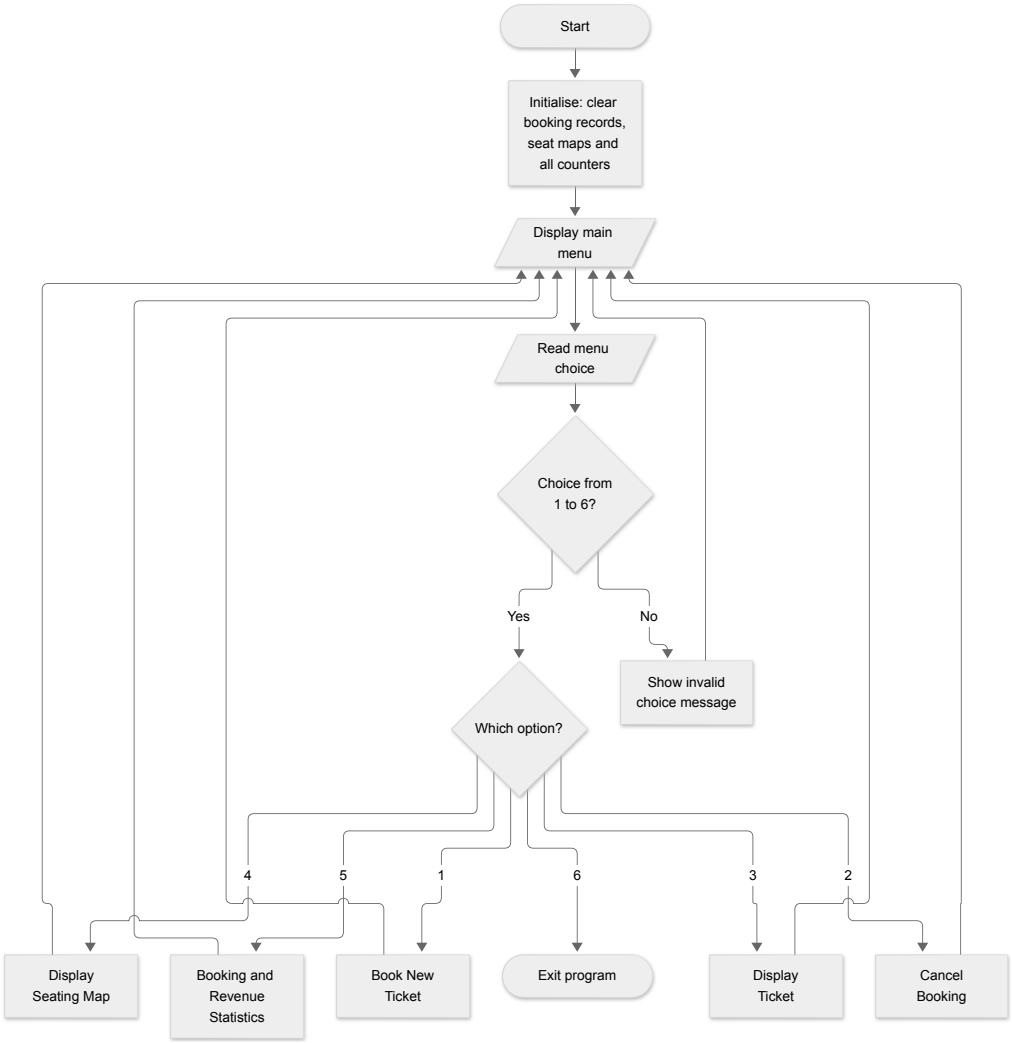



Figure 1: Main program flow: menu loop and dispatch to the six operations.

Options 3 and 4 are short read-only operations. **Display Ticket** reads a booking ID, searches the records for an active match, and either prints the formatted ticket or reports `"ERROR: Booking ID not found!"`. **Display Seating Map** reads a movie code and prints that movie's 20 seats, each as `[O]` or `[X]`, followed by the reserved count and the occupancy percentage.

### 2.3.2 Book New Ticket

Each field is validated as soon as it is entered, so an invalid value is rejected immediately and re-read rather than being carried forward. The flow is shown in three parts (Figures 2–4): identity fields, seat selection, and pricing/commit.

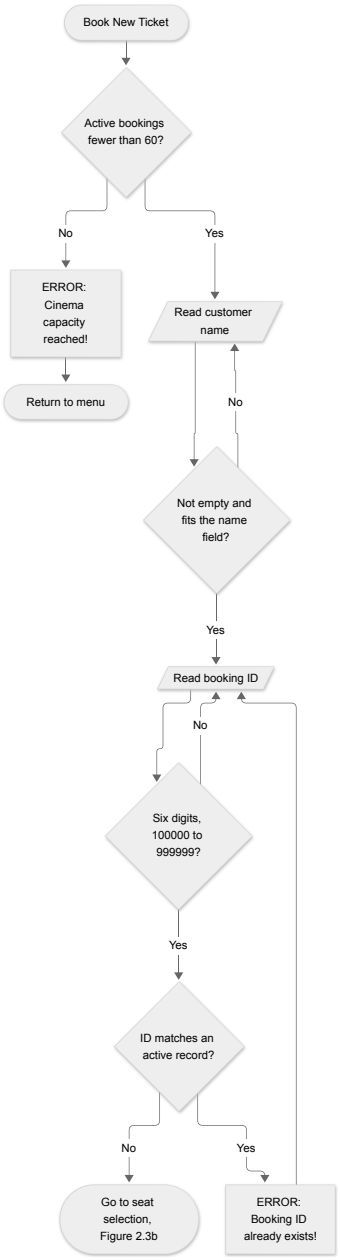



Figure 2: Book New Ticket, part 1: customer name and booking ID.

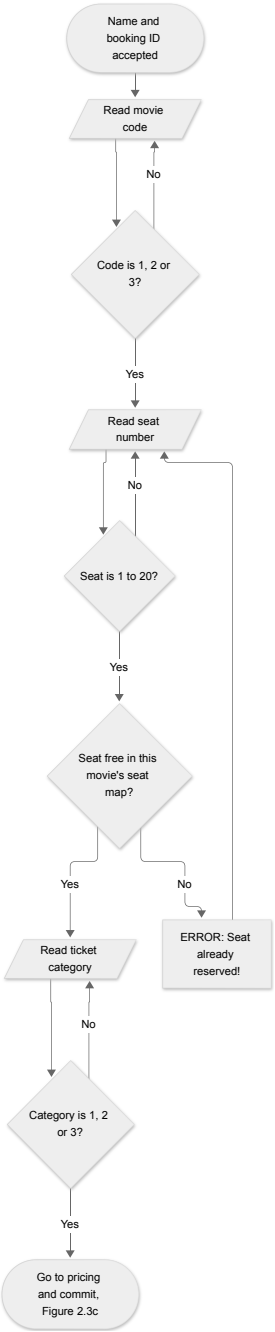



Figure 3: Book New Ticket, part 2: movie code, seat number, ticket category.

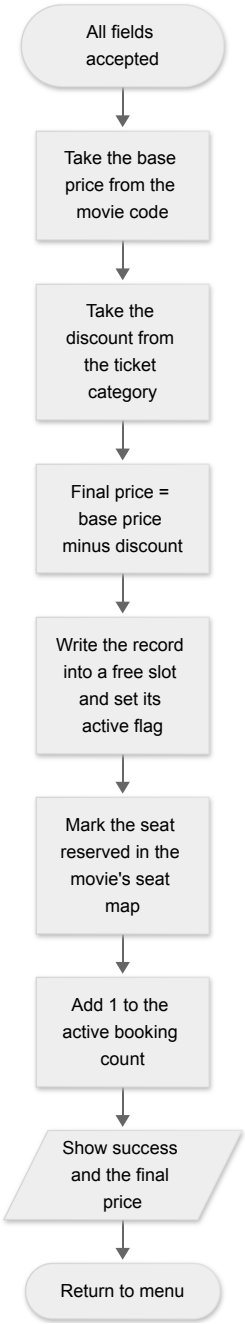



Figure 4: Book New Ticket, part 3: pricing and commit to the booking record.

### 2.3.3 Cancel Booking

Cancelling never deletes a record. It clears the record's active flag and releases the seat, which is what allows the same seat to be sold again in the same run.

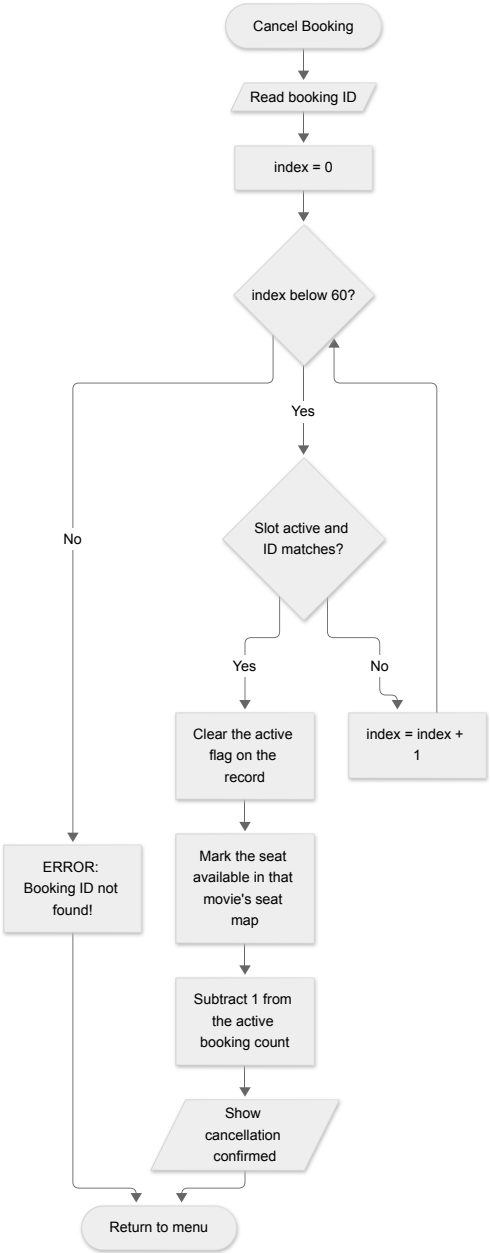



Figure 5: Cancel Booking: linear search over 60 slots, then clear the active flag and free the seat.

### 2.3.4 Booking and Revenue Statistics

Every figure in the report is recalculated from the records at the moment the option is chosen, so the report is always current and inactive records never contribute to it. The flow is shown in two parts: the scan over all 60 slots (Figure 6), and the derived figures and report (Figure 7).

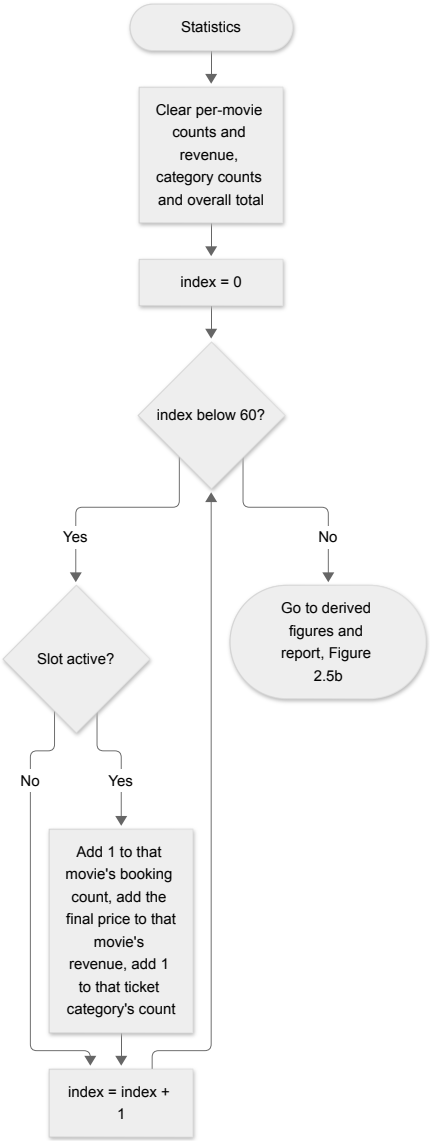



Figure 6: Statistics, part 1: scan the 60 record slots and accumulate per-movie and per-category counts.

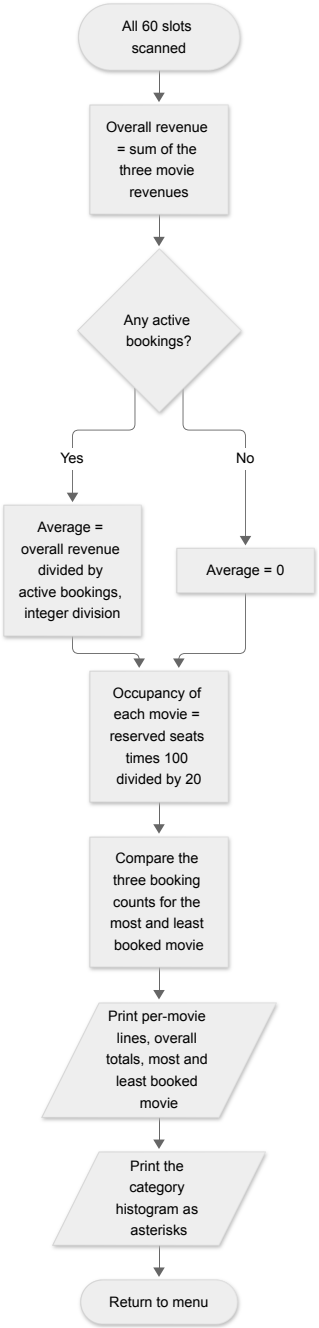



Figure 7: Statistics, part 2: derive overall totals, average, occupancy, and print the report.

### 2.3.5 Note on the repeated structures

Three of the four flows above repeat an action a known number of times over a fixed array: the duplicate-ID check and the record scan both sweep 60 record slots, and the seating map prints 20 seats. Chapter Two identified CX as the processor's dedicated counter register, used by default for counted repetition, so the problem's shape fits that register naturally. The validation branches are different in kind: each one repeats *until* the user supplies an acceptable value, so the number of repetitions is not known in advance. The project therefore requires both a counted repetition and a condition-controlled repetition, which satisfies the requirement for at least two loop structures.

## 2.4 Constraints

### 2.4.1 Memory

All storage is fixed in size and known before the program runs. Nothing is allocated while the program is running.

Each booking is stored as one fixed-size **record**: a single run of bytes holding every field for one ticket (name, ID, movie, seat, category, active flag, prices). The 60 records sit one after another in a fixed array, so a given booking is always found at the same offset: record index times record size. Table 1 breaks that record, and the system's other arrays, down byte by byte.

Table 1: Memory used by the main data structures.

| **Structure** | **Size calculation** | **Total** |
|---|---|---|
| Booking records | 60 records, 37 bytes each | 2,220 bytes |
| Seat maps | 3 movies, 20 seats, 1 byte per seat | 60 bytes |
| Per-movie booking counts | 3 doublewords | 12 bytes |
| Per-movie revenue totals | 3 doublewords | 12 bytes |
| Ticket category counts | 3 doublewords | 12 bytes |
| **Total main data** | | **about 2.3 KB** |

The 37-byte record is made up of a 21-byte name field (20 characters plus a null terminator), a 4-byte booking ID, three single bytes for movie code, seat number and category, one byte for the active flag, and two doublewords for the base price and final price.

### 2.4.2 Registers against memory

Chapter Two established that a processor has only a small, fixed set of registers, that they are fast but temporary, and that program data lives in main memory. CTRMS holds roughly 2.3 KB of state, so the records, the seat maps and the counters must all live in memory. Registers can only ever hold what is being worked on at that instant: the current array index, a loop counter, an address, or the result of one arithmetic step. This is the reason the system is built around arrays indexed by position rather than around values kept permanently in registers.

### 2.4.3 Capacity

- At most **60 active bookings** at any one time.
- Exactly **3 movies**, fixed, not configurable.
- Exactly **20 seats** per movie, so at most 60 seats in total across the cinema.
- Customer names limited to **20 characters**, stored in a 21-byte field with the null terminator.
- Booking IDs limited to the **900,000 values** from 100000 to 999999.

### 2.4.4 Integer arithmetic

The processor does provide floating-point registers, as covered in Chapter Two, but the project specification forbids floating-point instructions. Every calculation is therefore integer arithmetic, with these consequences:

- **Prices are exact.** Base prices are 8, 7 and 6 JOD and discounts are 0, 1 and 2 JOD, so every final price is a whole number from 4 to 8 JOD. No rounding occurs.
- **Occupancy is exact.** Occupancy is *reserved seats times 100 divided by 20*, which is the same as *reserved seats times 5*. Every value from 0 to 20 seats therefore maps to an exact percentage from 0 to 100, and nothing is lost to truncation.
- **The average ticket price is the only figure that loses precision.** It is *overall revenue divided by active bookings* in integer division, so the remainder is discarded. For example, 26 JOD over 4 bookings gives 6 JOD, not 6.5 JOD. The report shows the truncated value, which is what the specification requires.

### 2.4.5 Value ranges

Three range limits act as hard constraints on the design:

1. A booking ID can reach 999,999, which exceeds the 65,535 limit of a word, so IDs must occupy a doubleword.
2. Movie codes, seat numbers, categories and the active flag all stay within 0 to 255 and therefore fit in a single byte each.
3. The largest revenue the system can ever hold is 60 bookings at the top price of 8 JOD, which is **480 JOD**. This is nowhere near the doubleword limit, so no overflow handling is needed for any total.

Every quantity in the system (identifiers, counts, prices, revenue, percentages) is non-negative. All values are therefore treated as unsigned, and the two's-complement representation of negative numbers studied in Chapter One does not arise anywhere in this project.

### 2.4.6 Platform and course constraints

- The program is written in x86 assembly for **MASM with the Irvine32 library**, and runs as a 32-bit Windows console application in **protected mode**, the native mode described in Chapter Two.
- There is **no persistence**. All records live in memory only, so every booking is lost when the program exits.
- The program is **console only**. There is no graphical interface, no mouse input and no printing.
- The course imposes its own requirements that shape the design: at least **8 procedures**, at least **2 different loop structures**, IF, IF-ELSE and nested conditionals, and the use of all five required data types: strings, doublewords, bytes, characters and arrays.

## 2.5 Assumptions

1. **Seats belong to a movie, not to the cinema.** Seat 5 of Movie A and seat 5 of Movie B are different seats. A seat is unavailable only when that same movie and seat number are already reserved.

2. **Cancelled records stay in the array.** Cancelling clears the active flag; it does not erase the record or shift any other record. Only the active flag decides whether a record counts.

3. **Inactive records are invisible to every report.** They are excluded from the active booking count, from all revenue, from occupancy and from the category histogram.

4. **A freed slot and a freed seat are both reusable.** After a cancellation the record slot can hold a new booking and the seat can be sold again in the same run.

5. **A booking ID is unique among active bookings only.** Once a booking is cancelled, its ID may be used again by a new booking.

6. **One booking is one seat, one ticket and one customer.** There is no group booking and no multi-seat purchase in a single transaction.

7. **One user at a time.** The program is a single console session with no concurrent access, so no two bookings can ever be made at the same instant.

8. **Prices are fixed constants.** The three base prices and the three discounts are built into the program and are never entered or changed at runtime.

9. **All input arrives as ASCII text from the keyboard.** Where a number is expected, anything that is not a valid number in range is treated as invalid input and the prompt is repeated.

10. **The user is cinema staff, not the public.** The person at the keyboard is assumed to be trained in the menu, so prompts explain the legal range but no further help system is provided.

11. **All lab machines behave identically.** Every machine runs the same MASM and Irvine32 setup in protected mode, so no machine-specific behaviour is relied upon.

# Chapter Three
# Requirements Specification

## 3.1 Functional Requirements

The system must fulfill the following functional requirements to meet the project objectives. Priorities are classified as High (H), Medium (M), or Low (L).

Table 2: Functional Requirements

| ID | Description | Pri |
|---|---|---|
| FR01 | The system shall display a main menu with six options: Book, Cancel, Display Ticket, Display Seating Map, Statistics, and Exit. | H |
| FR02 | The system shall record new ticket bookings by accepting Customer Name, Booking ID, Movie Code, Seat Number, and Ticket Category. | H |
| FR03 | The system shall validate that the 6-digit Booking ID is unique and falls between 100000 and 999999. | H |
| FR04 | The system shall calculate final ticket prices by subtracting the applicable ticket category discount from the movie's base price. | H |
| FR05 | The system shall prevent duplicate seat reservations for the same movie hall. | H |
| FR06 | The system shall deactivate a booking and release its seat when a user cancels an active booking using its ID. | H |
| FR07 | The system shall format and display an individual ticket, including all pricing and discount details, when provided a valid active Booking ID. | M |
| FR08 | The system shall output a visual seating map for a specified movie, indicating available seats with `[O]` and reserved seats with `[X]`. | M |
| FR09 | The system shall calculate and display revenue statistics, total bookings, occupancy percentages, and a ticket category distribution histogram. | M |
| FR10 | The system shall restrict total active bookings to a maximum capacity of 60 reservations across the three movies. | H |

## 3.2 Non-Functional Requirements

The following constraints define the technical environment, structural quality, and architectural standards of the system.

Table 3: Non-Functional Requirements

| ID | Description |
|---|---|
| NFR01 | The software shall be implemented entirely in x86 Assembly Language using MASM. |
| NFR02 | The software shall utilize the Irvine32 library for all console input/output and string formatting operations. |
| NFR03 | The software shall rely exclusively on integer arithmetic for all pricing and statistical calculations; floating-point instructions are strictly forbidden. |
| NFR04 | The software shall use fixed-size memory records and arrays to manage the state of the up to 60 active booking entries. |

## 3.3 CLO Mapping

The specified requirements directly support the Course Learning Outcomes (CLOs) as defined in the project structure:

- **S1: Develop assembly language programs for microcomputers to perform basic input/output operations.**
  - **Mapping:** Supported by FR01, FR07, FR08, and FR09. The system fulfills this outcome by prompting the user for input, executing validation loops, and rendering highly formatted reports (menus, tickets, seating maps, and histograms) directly to the console[cite: 13].
- **S2: Implement and debug assembly language programs that perform arithmetic, string operations, searching, and memory manipulation tasks.**
  - **Mapping:** Supported by FR02, FR04, FR06, and NFR04. The logic explicitly requires iterating over arrays to perform searches (booking IDs, seat availability), reading strings (customer names), and executing integer arithmetic to calculate ticket pricing and cumulative statistics.
- **C1: Distinguish between different processor registers and use them appropriately in assembly language programming.**
  - **Mapping:** Supported by NFR01 and NFR03. The implementation mandates correct assignment and management of general-purpose registers (EAX, EBX, ECX, EDX, ESI, EDI) according to standard architectural conventions for calculations, pointers, loop counting, and library parameter passing.